

Full Provenance on the COSS Causal Axis: An Event-Level Closure of Causal Attribution and Confused-Deputy Exclusion

Nicola Gallo

27 September 2026

Abstract

Provenance Identity Continuity (PIC) was developed to answer a question that possession-based authorization leaves open: not only which authority an actor holds, but which execution occurrence may exercise it. PIC binds authority to execution occurrences in a causal history, relates adjacent occurrences by Proof of Relationship (PoR), and requires the carried authority to be non-expansive along that history. This paper asks whether that structure can serve as a general theory of security-state continuity, and uses it as the reference frame from which existing security models are read.

We develop Causally Ordered Security-State Theory (COSS) as the generalization of PIC. An occurrence carries a product state $\Sigma_i = (\sigma_i^{(j)})_{j \in J}$; each coordinate keeps its own refinement order, while PoR remains the single causal axis. Product Continuity then follows componentwise: ordinary causal transport cannot make any carried security state more permissive. PIC is recovered as the authority-only instance and relational PIC as the two-coordinate instance; lattice information flow, typed capability transfer, and decentralized labels can be placed as further coordinates without altering their semantics.

We then read Rajani, Garg, and Rezk’s 2016 formal study of confused-deputy attacks through this frame. Their full provenance semantics tracks explicit and implicit influence, and their Theorem 6 proves CDA-freedom with endorsement unconditionally for their region calculus. Within COSS, that influence is one instance of an influence relation on the same causal order used by PoR: we give a conservative event refinement whose erasure leaves their semantics unchanged, and we attribute to their work neither PoR, PoC, nor distributed lineage. The results of this paper rely only on three closure properties of the influence relation, of which full provenance is one realization.

Once all security-relevant effects and influence edges of the modeled machine are represented as events, causal attribution is derived by reachability inside the model. Combined with PIC authority continuity and effect admission, this yields a request-bounded effect theorem: in a closed COSS execution, an effect caused by a request can

exercise only authority present in that request’s origin, or introduced by an explicit authorized composition or relaxation event on its causal path, and the lineage-form confused-deputy condition is unsatisfiable along ordinary continuation. Effects and physical channels absent from the formal event model are not claimed to be covered.

1 Introduction

Security mechanisms answer different questions. A capability can answer whether an executor possesses authority over an object. An information-flow policy can answer whether information at one security class may flow to another. A relational authorization rule can restrict which kind of executor may become the next participant. A provenance rule can restrict which artifact, builder, or production history may be used. These are not merely different encodings of one scalar security property. They are different security state spaces.

The central proposal of this paper is to stop requiring one such state space to subsume the others. Instead, let each security system contribute one coordinate to the state of a concrete execution occurrence, and let causal continuity transport the product state.

Informally, if $S_1, \dots, S_m$ are security-state systems, an occurrence at step i carries

$$\Sigma_i = (\sigma_i^{(1)}, \dots, \sigma_i^{(m)}) \in \prod_{j=1}^m X_j, \quad (1)$$

where X_j is the state space of S_j . Every system supplies its own “no-more-permissive-than” relation $\preceq_j$. Ordinary causal continuation requires

$$\text{PoR}(s_i, s_{i+1}) \quad \wedge \quad \forall j, \sigma_{i+1}^{(j)} \preceq_j \sigma_i^{(j)}. \quad (2)$$

The first conjunct binds the states to one causal execution; the second prevents any security coordinate from expanding silently while that execution continues. At the event level, this is the lift from PIC’s $O \times R \times L$ to $\mathcal{B} \times L$: the causal axis is preserved, while the behavior judged at an occurrence may now carry facets for every security system in the product (Section 4.4).

This generalizes the continuity pattern already present in PIC. The first PIC model carries an authority context $C_i \subseteq O \times R$ and requires $C_{i+1} \subseteq C_i$ along a PoR-linked execution [17]. Its relational extension already replaces the scalar authority coordinate by the product state $(C_i, [\rho_i]_{\text{sem}})$ and orders that product componentwise by set inclusion and logical implication [18]. COSS takes the next step: the product itself is the general object, while authority is one possible coordinate.

The contribution is not the observation that security systems can be combined. The contribution is a minimal mathematical condition under which their states can be *continued together without collapsing their distinct semantics*: causal linkage plus componentwise non-expansion.

Positioning. PIC was developed independently, starting from the propagation of authority across distributed services and AI agents, and not from the language-based information-flow or object-capability literature. Its primitives, execution occurrences, an explicit lineage axis, receiver-verifiable PoR, non-expansive carried authority, and a successor that need not exist when its predecessor acts, were introduced for that setting [17, 18]. The present paper takes those primitives as the foundation and asks a further question: can the same continuity structure serve as a general theory, inside which established security models can be placed and their results reread? The readings of Bell–LaPadula, Denning, Boebert, Miller–Yee–Shapiro and Rajani–Garg–Rezk that follow are therefore interpretations made from the PIC/COSS frame. They do not claim that those authors used this frame, and they do not make PIC a derivation of their work.

The frame also covers settings outside the scope of those works: authority continuity across services and trust domains rather than within one program semantics, continuity that the receiving boundary can verify, successors that are unknown when a continuation is emitted, and a product of heterogeneous security coordinates carried along one causal history.

A second objective is to close a boundary left explicit in the earlier PIC series: causal attribution of an effect beyond an immediately witnessed handoff. Within COSS, this requires an influence relation on execution occurrences that represents every in-scope explicit and implicit dependency and is contained in the causal order. Rajani, Garg, and Rezk (RGR) provide a natural language-level instance of such a relation. Their full provenance semantics (FPs) tracks all value influence represented by their calculus, including implicit influence through branch and loop control, and proves CDA-freedom for the entire calculus without the program and heap restrictions required by their capability or explicit-only provenance results [9]. We read that influence as one realization of the COSS influence relation and place it on the same causal axis as PoR. The purpose is not to replace RGR’s theorem, but to show that, seen from PIC, full provenance and authority continuity are two views of one event structure

and can therefore be composed into a single causal proof.

Contributions. This paper develops ten results.

1. An explicit lift of PIC’s event space $O \times R \times L$ to a general security-event space $\mathcal{B} \times L$.
2. A single underlying causal event order on which authority, IFC, provenance, relational state, and other security views are attached to the same execution occurrences.
3. A general ordered security-state coordinate and its product construction, with a joint product-admission semantics.
4. A Product Continuity theorem showing that every ordinary descendant is origin-bounded in every coordinate and jointly in admitted behavior.
5. A per-coordinate continuity construction showing that PIC authority continuity and IFC/provenance continuity share PoR as the same causal basis; product continuity is their conjunction, not a second clock.
6. An application of the principal down-set embedding and a projection/factorization theorem showing what is lost when a decision omits a security coordinate or causal history.
7. A translated-continuity theorem for heterogeneous security-state spaces and a segment-continuity theorem for explicit authorized relaxations.
8. A PIC-based reading of RGR’s capability, explicit provenance, and full provenance results, faithful to their original statements, including their unconditional Theorem 6 for CDA-freedom with endorsement.
9. A conservative event refinement showing that RGR’s full-provenance influence is one instance of a COSS influence relation, causally ordered on the same occurrences as PoR; the derivation of causal attribution inside a closed event model depends only on the closure properties (E1)–(E3), not on RGR’s calculus.
10. A request-bounded effect theorem and a model-level exclusion of lineage-form confused-deputy states along ordinary continuation, with endorsement, composition, or authority relaxation represented as explicit authorized causal events.

2 Rajani–Garg–Rezk: Full Provenance and CDA Freedom

We summarize RGR’s results because they give an independent, language-based treatment of the confused deputy that can be read inside the PIC/COSS frame. The summary is descriptive and follows their statements; the continuity and attribution results of this paper are

stated over the COSS event structure and do not depend on their calculus.

Rajani, Garg, and Rezk study access control, object-capability semantics, and confused-deputy attacks in a small imperative *region calculus* [9]. Their formal language contains principals, mutable references, read and write capabilities R_r and W_r , conditionals, loops, assignments, sequencing, and commands executed inside regions. A principal lattice supplies the write-integrity policy through an ownership map O from references to principals.

Their first comparison equips the same calculus with two operational semantics. Access-control semantics checks authority when a reference is written. Capability semantics implements Miller et al.’s Property A, “no designation without authority,” by preventing a region from obtaining a write capability that it is not authorized to wield. RGR prove that the access-control semantics is strictly more permissive than this capability semantics, while both satisfy the write-integrity property they formulate.

The paper’s second contribution is an extensional characterization of confused-deputy freedom. An authority context E_{ρ_A} contains a hole in an adversary region ρ_A , and an attacker-interest set AIS identifies the references of concern. Informally, a reference is CDA-safe if either the adversary cannot control its final value or the adversary could have written that final value directly. Endorsed regions are then excluded from the attack condition because an endorsed principal explicitly chooses to act on lower integrity influence. RGR denote the resulting property by $\text{CDAF-E}(E_{\rho_A}, H, \rightarrow_{red})$.

This definition lets RGR separate explicit from implicit designation. Their capability semantics blocks the classical case in which the attacker supplies the privileged reference itself, but it does not block all cases in which the deputy already possesses the privileged reference and the attacker instead controls the value written or the control path that determines the write. Their Theorem 4 therefore proves CDA-freedom for capability semantics only under strong restrictions on programs and the initial heap [9].

RGR then add provenance tracking to the access-control baseline. Their explicit-only provenance semantics (EPs) labels each computed value by a principal that lower-bounds the principals whose references have explicitly influenced the value. EPs deliberately ignores implicit control-flow influence, and their Theorem 5 still needs restrictions, although weaker ones than capability semantics.

Their full provenance semantics (FPs) removes those remaining restrictions. Every computed value still carries a principal-valued provenance label. In addition, the semantic state carries a stack PC of program-counter labels. On entry to an **if** or **while** body, the new program-counter label is the meet of the current pc and the label of the condition; on exit, the old pc is restored. The FPs assignment rule checks the provenance of the target,

the value, and the current pc against the owner of the reference being written. Thus explicit data influence and implicit control influence are both part of the enforcement semantics [9].

The key result is their Theorem 6:

$$\text{CDAF-E}(E_{\rho_A}, H, \rightarrow_{FP}) \text{ holds unconditionally.} \quad (3)$$

“Unconditionally” here is relative to the complete RGR region calculus and FPs semantics: unlike their capability and EP results, no additional program or heap restrictions are assumed. The proof route is also explicit. FPs establishes a strong non-interference/information-flow-integrity property, and RGR’s Theorem 7 proves that non-interference with endorsement implies CDA-freedom with endorsement [9].

Two boundaries matter for the present paper. First, RGR’s FPs labels are *principal-valued abstractions of influence*; their paper does not attach event identifiers to labels. Second, RGR do not define PoR, PoC, a distributed lineage protocol, or a Lamport-style event graph. The event construction below is therefore a new conservative refinement of the influence already represented by FPs, not a claim about terminology or machinery present in RGR.

3 Historical Motivation: Different Security Coordinates

Bell and LaPadula developed a mathematical model of secure computer states and rules intended to preserve security under state transitions [1, 2]. Denning subsequently formulated secure information flow using a lattice of security classes, giving a general mathematical view of systems that restrict information flow [3].

Capability systems start from a different primitive. Authority is represented by protected designators that permit operations on objects. In 1984, Boebert showed that an *unmodified* capability machine, in the absence of additional mechanisms, cannot enforce the Bell–LaPadula *-property in his model [4]. His concrete attack lets a low-clearance program place a read/write capability for a low object into that low object; a later high-clearance program may read the low object, obtain the capability, and then combine read access to high data with write access to the low object. Boebert’s concluding explanation is that, in the pure capability machine he defines, the right to exercise access carries the ability to propagate that access.

Miller, Yee, and Shapiro revisited this construction directly in *Capability Myths Demolished* [5]. They agree that the particular construction described by Boebert fails to enforce the *-property, but argue that the attack assumes capabilities can be transmitted wherever data can be transmitted. In partitioned or type-enforced capability systems, capability transfer and data transfer are distinguishable operations. Their two-level construction

independently controls those operations and is presented as a counterexample to the broader claim that capability systems as a class cannot enforce the $\ast$ -property. The same paper also stresses that a static access matrix is not a complete description of a security mechanism: the rules by which access relationships evolve over time materially affect the security properties of the resulting system.

Miller and Shapiro develop a complementary argument in *Paradigm Regained* [6]. They distinguish permission from authority and explicitly make program behavior on permitted causal pathways part of authority analysis. Their object-capability model carries overt causation by messages over permitted paths. Their examples use Redell’s Caretaker pattern for revocation, factories for overt confinement, and a data diode for the $\ast$ -properties. They also show how multiple independently motivated abstractions can coexist over the same graph under “mutually suspicious composition.”

These papers therefore already contain dynamic, behavioral, and causal reasoning; COSS does not claim otherwise. The distinction made here is representational. In those analyses, causal pathways and abstraction behavior are derived from the reference graph and program semantics. COSS instead makes causal lineage an explicit axis of the security event and transports an ordered security state along that axis. Nor does COSS assume that a capability implementation can contribute only an authority coordinate: a capability design may itself mediate information flow, transfer type, request context, or another security distinction. The general claim is narrower: if an enforcement decision is invariant under a distinction that changes the target security verdict, that decision cannot enforce the target property. A typed transfer rule or access abstraction may expose exactly the distinction that a coarser model omitted.

This paper does *not* claim that Boebert, Bell–LaPadula, Denning, Miller, or Shapiro formulated their work in this product language. The product view is a new interpretation intended to explain why apparently conflicting results can all be correct under different observable state spaces.

4 Preliminaries

4.1 Execution occurrences and the common causal order

Let S be the set of concrete execution occurrences of one modeled run. The machine may be sequential, concurrent, or distributed. Let

$$\rightarrow_c \subseteq S \times S \quad (4)$$

be an acyclic immediate causal-predecessor relation, and let $\prec_c$ denote its transitive closure. In a sequential reduction trace, $\rightarrow_c$ is a chain. In a concurrent or distributed execution it is generally a DAG, in the sense of a causal happened-before style order rather than a requirement of one global clock [15]. This causal order is the underlying

order of the run; security mechanisms do not receive independent physical clocks merely because they use different state spaces. We require the semantic causal order to be *dependency adequate*: whenever an occurrence consumes a value, message, or state produced by an earlier occurrence, or its reachability is control-dependent on a condition evaluated at an earlier occurrence, the producer/condition is an ancestor of the consumer/controlled occurrence under $\prec_c$. This is a local semantic condition on causal edges; it does not assume the request-to-effect Causal Attribution theorem proved later.

PoR is the receiver-verifiable immediate-continuation relation used by PIC and COSS. At the semantic level considered here, sound PoR satisfies

$$\text{PoR}(s, s') \implies s \rightarrow_c s'. \quad (5)$$

For a closed semantic execution, every immediate causal edge in the modeled security-relevant event graph is required to be represented by a PoR-witnessable transition. A concrete realization may establish this relation using signatures, attestations, observed handoffs, trusted-runtime evidence, or an information-flow/provenance witness. Evidence and the relation evidenced are kept distinct.

A finite *lineage path* is

$$\pi = \langle s_0, s_1, \dots, s_n \rangle \quad (6)$$

with $\text{PoR}(s_i, s_{i+1})$ at every adjacent step. Let L denote the set of such finite causal paths. In a DAG, several lineage paths may share an origin or endpoint; the underlying occurrence $s \in S$ is unique even when it belongs to several paths. The path notation is retained because the basic continuity proof is transitivity along a causal path.

4.2 One occurrence, several security views

An occurrence s_i may simultaneously carry authority, information-flow, provenance, identity, relational, geographic, or other security observations. These coordinates may inhabit unrelated mathematical spaces, but they are observations of the same event. For example,

$$\omega_i = (\ell_i, \iota_i, g_i, \Sigma_i), \quad (7)$$

where ℓ_i is causal position or lineage information, ι_i is an executor or identity observation, g_i is a geographic or network location, and Σ_i is the security state carried by the occurrence.

Lineage grows by causal extension rather than attenuation, while the current identity or location may vary from step to step. If geography is a security restriction, the monotone object is instead an admissible set or predicate, e.g.

$$g_i \in G_i^{\text{allow}}, \quad G_{i+1}^{\text{allow}} \subseteq G_i^{\text{allow}}. \quad (8)$$

This distinction prevents the causal coordinates of an occurrence from being confused with the policy state that constrains them.

4.3 Per-coordinate continuity over the same PoR

For a carried coordinate j , define its path-continuity property by

$$\text{PoC}_j(\pi) \iff \bigwedge_{i=0}^{n-1} \left(\text{PoR}(s_i, s_{i+1}) \wedge \sigma_{i+1}^{(j)} \preceq_j \sigma_i^{(j)} \right). \quad (9)$$

The authority-only instance is the original PIC/PoC condition $C_{i+1} \subseteq C_i$. An IFC coordinate has its own refinement order, while full provenance contributes an influence relation on the same occurrences (Section 10). Product continuity is therefore not a product of independent times. It is the conjunction of several continuity disciplines all sharing the same PoR edges:

$$\text{PoC}_{\text{prod}}(\pi) \iff \bigwedge_{j \in J} \text{PoC}_j(\pi). \quad (10)$$

This formalizes the distinction between the common causal substrate and the coordinate-specific security invariants transported over it.

4.4 From $O \times R \times L$ to $\mathcal{B} \times L$

In PIC an authority event is an occurrence of a privilege within a causal lineage,

$$e = (o, r, \ell) \in O \times R \times L, \quad (11)$$

and the state carried by an occurrence is an authority context $C \subseteq O \times R$ [17]. The privilege coordinates $O \times R$ say *what* is exercised; the lineage coordinate L says *in which causal history* it is exercised. COSS keeps the lineage axis unchanged and replaces the single privilege universe by the product of the behavior universes of all carried security systems.

Anticipating Definition 2, let each carried security system contribute a behavior universe $\mathcal{B}_j$.

Definition 1 (Security event). *Let $\mathcal{B} = \prod_{j \in J} \mathcal{B}_j$. A security event is*

$$e = (b, \ell) \in \mathcal{B} \times L, \quad (12)$$

where ℓ is the causal history of the occurrence and $b = (b^{(j)})_{j \in J}$ collects, for each coordinate, the facet of the occurrence that the coordinate judges: the privilege it exercises, the flow it performs, the executor transition it realizes, or the location at which it occurs.

For the single authority coordinate, $\mathcal{B}_{\text{auth}} = O \times R$ and $\mathcal{B} \times L = O \times R \times L$: PIC's event space is the one-coordinate instance. The same substitution acts on states, where $\mathcal{P}(O \times R)$ becomes $X = \prod_j X_j$. The construction of PIC is otherwise transported unchanged: states are carried along PoR-linked occurrences, and ordinary continuation may only refine them.

Remark 1. *If a coordinate imposes nothing on a given occurrence, its facet may be a distinguished element $\perp_j \in \mathcal{B}_j$ admitted by every state of that coordinate.*

5 Ordered Security-State Systems

Definition 2 (Security-state coordinate). *A security-state coordinate is a tuple*

$$\mathfrak{S}_j = (X_j, \preceq_j, \mathcal{B}_j, \llbracket \cdot \rrbracket_j), \quad (13)$$

where:

- X_j is a set of security states;
- $\preceq_j$ is a partial order interpreted as “no more permissive than”;
- $\mathcal{B}_j$ is a universe of behaviors, transitions, principals, flows, or other semantically admitted possibilities; and
- $\llbracket x \rrbracket_j \subseteq \mathcal{B}_j$ is the semantic extension admitted by state $x \in X_j$.

The coordinate is semantically monotone when

$$x' \preceq_j x \implies \llbracket x' \rrbracket_j \subseteq \llbracket x \rrbracket_j. \quad (14)$$

The definition deliberately does not require every security system to be born as a powerset. A lattice label, a logical predicate, a capability set, a provenance constraint, or a history-dependent state can all be used if they admit a sound refinement order.

Remark 2 (Coordinates are semantic dimensions). *A coordinate is not required to correspond one-to-one with an implementation mechanism. One mechanism may realize several security coordinates, as when a type-enforced capability runtime distinguishes authority from data transfer and capability transfer; conversely, several mechanisms may jointly realize one coordinate. COSS reasons over the security distinctions exposed to the admission decision, not over product names or implementation categories.*

Remark 3 (Refinement is not a complete update semantics). *The order $\preceq_j$ is the condition needed for the continuity theorem, not a claim that every refinement step is reachable or permitted by the concrete security system. A coordinate may impose a stricter local transition relation $U_j \subseteq X_j \times X_j$ satisfying*

$$U_j(x, x') \implies x' \preceq_j x. \quad (15)$$

All ordinary-continuation results below remain valid when transitions must satisfy both U_j and the refinement order. This leaves room for the richer update rules that distinguish dynamic security mechanisms even when they share the same static state representation.

Definition 3 (Product security state). *For a finite family J of security-state coordinates, define*

$$X = \prod_{j \in J} X_j, \quad \Sigma = (\sigma^{(j)})_{j \in J}. \quad (16)$$

The product refinement order is

$$\Sigma' \preceq \Sigma \iff \forall j \in J, \sigma'^{(j)} \preceq_j \sigma^{(j)}. \quad (17)$$

For a concrete execution, let

$$\text{st} : S \rightarrow X \quad (18)$$

assign exactly one product security state to each execution occurrence, and write $\Sigma_i := \text{st}(s_i)$. If $\ell \in L$ is a finite causal history with endpoint s_n , define

$$\Sigma(\ell) := \text{st}(s_n). \quad (19)$$

Thus the state used by an event is a well-defined function of the endpoint occurrence of its represented history.

Proposition 1 (Product order). *If every $(X_j, \preceq_j)$ is a partial order, then $(X, \preceq)$ defined by (17) is a partial order.*

Proof. Reflexivity, antisymmetry, and transitivity hold componentwise.

Remark 4 (Coordinates need not be independent). *The Cartesian product is an ambient state space, not an assumption that every combination of coordinate states is meaningful. If cross-coordinate constraints permit only states in $Y \subseteq X$, the induced order on Y is still a partial order and Product Continuity applies unchanged to lineages whose states remain in Y . Likewise, if only certain combinations of behavior facets are meaningful, one may restrict the event universe to $\mathcal{B}^* \subseteq \prod_j \mathcal{B}_j$ and interpret product admission on $\mathcal{B}^* \cap \prod_j \llbracket \sigma^{(j)} \rrbracket_j$.*

6 Causally Ordered Security-State Continuity

Definition 4 (Valid COSS transition). *A transition*

$$(s_i, \Sigma_i) \longrightarrow (s_{i+1}, \Sigma_{i+1}) \quad (20)$$

is COSS-valid when

$$\text{PoR}(s_i, s_{i+1}) \quad \wedge \quad \Sigma_{i+1} \preceq \Sigma_i. \quad (21)$$

Equivalently, every security coordinate refines or preserves its predecessor state while the execution advances along a verified causal edge.

Definition 5 (Product security-state continuity). *A lineage*

$$\Pi = \langle (s_0, \Sigma_0), \dots, (s_n, \Sigma_n) \rangle \quad (22)$$

has product security-state continuity when every adjacent transition is COSS-valid.

Theorem 1 (Product Continuity). *If Π has product security-state continuity, then*

$$\Sigma_n \preceq \Sigma_{n-1} \preceq \dots \preceq \Sigma_0. \quad (23)$$

Hence, for every coordinate j ,

$$\sigma_n^{(j)} \preceq_j \sigma_0^{(j)}. \quad (24)$$

If the coordinate is semantically monotone, then

$$\llbracket \sigma_n^{(j)} \rrbracket_j \subseteq \llbracket \sigma_0^{(j)} \rrbracket_j. \quad (25)$$

Proof. Every valid transition gives $\Sigma_{i+1} \preceq \Sigma_i$. Transitivity of the product order gives $\Sigma_n \preceq \Sigma_0$. Projection to coordinate j gives (24); semantic inclusion follows from (14).

The theorem states the general continuity invariant for lineages containing only ordinary continuations: causal transport cannot introduce a possibility forbidden by the origin state of any carried security system.

Theorem 2 (Pathwise Product Continuity on a causal DAG). *Let $q, e \in S$ with $q \prec_c e$. If every edge of a causal path from q to e is an ordinary COSS transition, then*

$$\text{st}(e) \preceq \text{st}(q). \quad (26)$$

If every immediate predecessor edge of a multi-parent ordinary join is required to satisfy the same componentwise non-expansion rule, the bound holds along every parent path.

Proof. Choose a causal path $q = s_0 \rightarrow_c s_1 \rightarrow_c \dots \rightarrow_c s_k = e$. Each ordinary edge gives $\text{st}(s_{i+1}) \preceq \text{st}(s_i)$. Transitivity yields $\text{st}(e) \preceq \text{st}(q)$. The multi-parent statement follows because the same argument applies independently to every immediate-predecessor path.

Definition 6 (Product admission). *The product state $\Sigma = (\sigma^{(j)})_{j \in J}$ admits $b \in \mathcal{B}$ iff $b^{(j)} \in \llbracket \sigma^{(j)} \rrbracket_j$ for every j . Write*

$$\llbracket \Sigma \rrbracket := \prod_{j \in J} \llbracket \sigma^{(j)} \rrbracket_j \subseteq \mathcal{B}. \quad (27)$$

Proposition 2 (Joint semantic monotonicity). *If every coordinate is semantically monotone, then $\Sigma' \preceq \Sigma$ implies $\llbracket \Sigma' \rrbracket \subseteq \llbracket \Sigma \rrbracket$. Consequently, under product security-state continuity, $\llbracket \Sigma_n \rrbracket \subseteq \llbracket \Sigma_0 \rrbracket$.*

Proof. $\Sigma' \preceq \Sigma$ gives $\sigma'^{(j)} \preceq_j \sigma^{(j)}$ for every j , hence $\llbracket \sigma'^{(j)} \rrbracket_j \subseteq \llbracket \sigma^{(j)} \rrbracket_j$ by semantic monotonicity, and a product of inclusions is an inclusion. The second claim follows from Product Continuity.

A later occurrence therefore admits only behaviors that the origin state admits in every coordinate jointly, not merely coordinate by coordinate.

7 Why Refinement Can Be Read as Set Inclusion

The previous theorem is intentionally stated over partial orders rather than subsets. Nevertheless, the subset intuition can be recovered for every partial order.

Definition 7 (Principal lower-set representation). *For a poset $(X, \preceq)$ define*

$$\downarrow x = \{y \in X \mid y \preceq x\}. \quad (28)$$

Theorem 3 (Principal down-set embedding). *For every poset $(X, \preceq)$, the standard principal down-set map*

$$\eta : X \rightarrow \mathcal{P}(X), \quad \eta(x) = \downarrow x \quad (29)$$

is an order embedding:

$$x' \preceq x \iff \downarrow x' \subseteq \downarrow x. \quad (30)$$

Proof. If $x' \preceq x$ and $y \preceq x'$, transitivity gives $y \preceq x$, so $\downarrow x' \subseteq \downarrow x$. Conversely, $\downarrow x' \subseteq \downarrow x$ implies $x' \in \downarrow x$ because $x' \in \downarrow x'$ by reflexivity. Hence $x' \preceq x$.

This standard principal down-set embedding formalizes the subset intuition behind the present work. A security system need not literally store a set, but its refinement order can be represented faithfully as inclusion among sets of states no more permissive than the current one. This inclusion must not be confused with semantic admission: $\downarrow x \subseteq X$ is a set of *states*, whereas $\llbracket x \rrbracket \subseteq \mathcal{B}$ is a set of *behaviors*. Semantic monotonicity connects refinement to behavior inclusion, but the two constructions are distinct. The familiar PIC rule $C_{i+1} \subseteq C_i$ is therefore one concrete instance of a more general ordered refinement pattern.

8 Projection, Quotients, and Coordinate Blindness

The second PIC paper interprets lineage-forgetting as a quotient and shows that lineage-invariant decisions factor through the projection that forgets causal history [18]. The same construction applies to arbitrary products of security-state coordinates.

Let

$$X = \prod_{j \in J} X_j \quad (31)$$

and let $K \subseteq J$. Define the coordinate projection

$$p_K : X \rightarrow X_K := \prod_{j \in K} X_j. \quad (32)$$

Definition 8 (K -invariant decision). *A decision rule $A : X \rightarrow \{0, 1\}$ is K -invariant when*

$$p_K(x) = p_K(y) \implies A(x) = A(y). \quad (33)$$

Theorem 4 (Projection Factorization). *A decision rule $A : X \rightarrow \{0, 1\}$ is K -invariant if and only if there exists a unique rule*

$$\bar{A} : p_K(X) \rightarrow \{0, 1\} \quad (34)$$

such that

$$A = \bar{A} \circ p_K. \quad (35)$$

Proof. If $A = \bar{A} \circ p_K$, equal projections yield equal decisions. Conversely, define $\bar{A}(z) = A(x)$ for any x with $p_K(x) = z$. K -invariance makes the definition independent of the representative. Uniqueness follows because p_K is surjective onto $p_K(X)$.

Corollary 1 (Omitted-coordinate impossibility). *Suppose a target security property $P : X \rightarrow \{0, 1\}$ distinguishes two states $x, y \in X$ for which $p_K(x) = p_K(y)$. No K -invariant decision rule can equal P on all of X .*

Proof. Every K -invariant rule must give the same result on x and y , while P requires different results.

This is the general form of the earlier lineage-forgetting observation: a security decision cannot enforce a distinction that its observation function erases.

Corollary 2 (Behavioral-state blindness). *Let $X = X_{\text{prot}} \times X_{\text{beh}}$ separate a primitive protection-state coordinate from a coordinate describing security-relevant behavior or an access abstraction. If a target property $P : X \rightarrow \{0, 1\}$ satisfies*

$$P(p, h_1) \neq P(p, h_2) \quad (36)$$

for some fixed $p \in X_{\text{prot}}$, then no decision rule that factors through the projection $(p, h) \mapsto p$ can implement P on all of X .

Proof. This is Omitted-coordinate Impossibility with $K = \{\text{prot}\}$.

This corollary gives an order-independent statement of the blind spot discussed by Miller and Shapiro: if two systems have the same protection graph but the behavior of a security-enforcing abstraction changes the authority or flow verdict, an analysis that observes only the protection graph is necessarily too coarse for that distinction [6]. It does not imply that protection-state analysis is unsound when it succeeds; it states only that such a projection cannot reproduce a behavior-sensitive property in general.

Using the endpoint-state assignment of (19), the COSS admission rule for an event (b, ℓ) is

$$A_{\text{COSS}}(b, \ell) = 1 \iff b \in \llbracket \Sigma(\ell) \rrbracket. \quad (37)$$

Let $q_L : \mathcal{B} \times L \rightarrow \mathcal{B}$, $(b, \ell) \mapsto b$, forget the lineage.

Corollary 3 (Lineage-forgetting blindness in COSS). *Let $b \in \llbracket \Sigma(\ell) \rrbracket \setminus \llbracket \Sigma(\ell') \rrbracket$. No decision rule that factors through q_L equals A_{COSS} on both (b, ℓ) and (b, ℓ') .*

Proof. $q_L(b, \ell) = q_L(b, \ell')$, so every rule factoring through q_L gives both events the same verdict, while $A_{\text{COSS}}(b, \ell) = 1$ and $A_{\text{COSS}}(b, \ell') = 0$. This is the same fiber argument as Projection Factorization, now applied to the projection that omits L from the event space.

For the authority coordinate alone this is the lineage-invariance result of PIC [17]. In COSS it holds for every carried security system at once: the same behavior can be admissible in one causal history and inadmissible in another, in authority, in information flow, in relational admissibility or in any other coordinate, and a decision that forgets the history cannot tell the two apart.

8.1 A three-coordinate example

Consider a three-step agent lineage

$$s_0 \rightarrow s_1 \rightarrow s_2 \quad (38)$$

carrying authority, information-flow, and geographic coordinates

$$\Sigma_i = (C_i, F_i, G_i), \quad \Sigma_2 \preceq \Sigma_1 \preceq \Sigma_0. \quad (39)$$

At s_2 , suppose C_2 still permits the operation (send,reportAPI) and G_2 admits the agent's current EU execution region, but F_2 excludes the flow facet f_{ext} representing transmission of the current data to an external sink. Let b be the event facet tuple whose authority component is that send operation, whose flow component is f_{ext} , and whose geographic component is the current EU region. Authority and geography admit their respective facets, but IFC does not, so

$$b \notin \llbracket \Sigma_2 \rrbracket \quad (40)$$

and the product rejects the event. An authority-only decision would admit the event even though the complete product state rejects it: the authority coordinate itself remains valid, while the IFC coordinate does not.

Now consider a different causal history ℓ' carrying the same authority and geographic states but an IFC state F' that admits f_{ext} , for example after a separately authorized declassification. If ℓ denotes the first history ending at s_2 , the same event facet tuple b may satisfy

$$b \in \llbracket \Sigma(\ell') \rrbracket \quad \text{while} \quad b \notin \llbracket \Sigma(\ell) \rrbracket. \quad (41)$$

The example exhibits both claims of the theory: all relevant security coordinates constrain one causal occurrence jointly, and a decision that forgets the lineage cannot distinguish histories whose current admissibility differs.

9 Classical Security Models as Coordinates

9.1 Authority and capabilities

Let O be operations and R resources. Define

$$X_{\text{auth}} = \mathcal{P}(O \times R), \quad C' \preceq_{\text{auth}} C \iff C' \subseteq C. \quad (42)$$

This coordinate says which authority may participate in the current causal execution. It does not identify the executor that happens to possess other ambient authority.

With this coordinate alone, COSS-validity reduces to

$$\text{PoR}(s_i, s_{i+1}) \quad \wedge \quad C_{i+1} \subseteq C_i, \quad (43)$$

which is precisely the abstract continuity rule of PIC [17].

This specialization is compatible with the stronger object-capability results in the literature. *Capability Myths Demolished* identifies absence of ambient authority,

inseparability of designation and authority, and access-controlled delegation channels as important properties of object-capability systems; it also emphasizes that authority conveyed in the context of a request helps avoid the classical confused-deputy pattern [5]. COSS does not treat those protections as missing. A capability system may realize several security distinctions at once; the lineage coordinate matters only for a target property whose verdict also varies with causal execution after the capability-level distinctions have been fixed.

9.2 Information-flow admissibility

Let D be a domain of information sources and sinks, security classes, or other flow endpoints. One possible semantic IFC coordinate is the set of currently admitted flows

$$F \subseteq D \times D. \quad (44)$$

Define

$$F' \preceq_{\text{ifc}} F \iff F' \subseteq F. \quad (45)$$

A Bell-LaPadula or lattice policy can induce such an admissible-flow relation; this paper does not replace the internal semantics of those models. It treats their effective admissibility state as an independent coordinate whose ordinary continuation must not become more permissive.

An alternative is to take lattice labels or complete IFC policy states as X_{ifc} and define $\preceq_{\text{ifc}}$ directly from their semantic flow sets. Equation (45) is therefore an example, not a claim that all IFC implementations literally store F .

The policy state F_i should be distinguished from the dynamic provenance of a particular run. The former answers which flows are admissible; the latter answers which earlier occurrences actually influenced a value or control decision in this execution. RGR's FPs supplies a formal influence abstraction through value labels and the program-counter stack. Section 10 lifts that influence to event resolution without changing the original FPs accept/reject behavior.

9.3 Typed data and capability transfer

The distinction used by Miller, Yee, and Shapiro to revisit Boebert can itself be represented as an ordered security coordinate without claiming that their systems used this notation. Let Λ be a set of security domains and let

$$K = \{\text{data}, \text{cap}\}, \quad \mathcal{B}_{\text{tr}} = \Lambda \times \Lambda \times K \quad (46)$$

classify a transfer by source domain, destination domain, and whether the transferred value is ordinary data or a capability. Define

$$X_{\text{tr}} = \mathcal{P}(\mathcal{B}_{\text{tr}}), \quad (47)$$

$$Q' \preceq_{\text{tr}} Q \iff Q' \subseteq Q, \quad \llbracket Q \rrbracket_{\text{tr}} = Q. \quad (48)$$

The coordinate can therefore admit a data transfer between two levels while rejecting a capability transfer over the same pair of levels.

This captures the specific distinction made in *Capability Myths Demolished*: Boebert’s attack requires a high-clearance subject to obtain, by reading a low object, a capability that confers write access to that low object; the later type-enforced construction distinguishes capability movement from data movement and blocks the cross-level capability transfer [5]. The coordinate Q is only an illustrative semantic encoding of that distinction, not a claim that an object-capability implementation literally stores a set of typed flows.

9.4 Relational admissibility

The relational extension of PIC carries a predicate ρ_i over successor/predecessor profiles and orders predicates by implication [18]. After quotienting by semantic equivalence, define

$$X_{\text{rel}} = \text{Pred}(\mathcal{T}_J) / \equiv_{\text{sem}}, \quad (49)$$

with

$$[\rho'] \preceq_{\text{rel}} [\rho] \iff \rho' \Rightarrow \rho. \quad (50)$$

Semantically,

$$\rho' \Rightarrow \rho \iff \llbracket \rho' \rrbracket \subseteq \llbracket \rho \rrbracket. \quad (51)$$

Thus the existing PIC product

$$\Gamma_i = (C_i, [\rho_i]_{\text{sem}}) \quad (52)$$

is already a two-coordinate instance of COSS.

9.5 Geographic, provenance, and product constraints

A geographic restriction can be represented by an admissible region set G_i^{allow} ordered by inclusion, while the observed current location g_i is checked for membership. A software-product or provenance coordinate can analogously represent the set of admissible artifact identities, builders, attestations, or derivation histories. These coordinates do not need to share an ontology with authority or IFC; they need only expose a sound refinement order.

This distinction is useful for invocation. A product can participate in many runtime invocations, just as one executor can realize many execution occurrences. Product identity is therefore not itself an invocation. A future DAG extension can treat invocation as a causal join between an authority lineage and a product/provenance lineage while preserving the coordinates that justify each parent.

10 Full Provenance on the COSS Causal Axis

COSS needs an influence relation on execution occurrences: a relation that records which earlier occurrences

influence the data or control of later ones. Any relation satisfying the closure properties (E1)–(E3) of Definition 10 serves. RGR’s full provenance semantics is a natural language-level instance, and this section shows how it fits. FPs labels values and the current control context with principal-valued integrity information, whereas COSS relates concrete execution occurrences and transports security state along them. The connection is obtained by refining an FPs reduction trace with ghost event dependencies and then erasing those dependencies back to the original FPs semantics.

10.1 Conservative event refinement of FPs

Consider an RGR FPs execution

$$\Theta_0 \rightarrow_{\text{FP}} \Theta_1 \rightarrow_{\text{FP}} \cdots \rightarrow_{\text{FP}} \Theta_n, \quad (53)$$

where each Θ_i contains the heap, the PC stack, and the current program/command state as in RGR. Associate a distinct occurrence $e_i \in S$ with each reduction step $\Theta_i \rightarrow_{\text{FP}} \Theta_{i+1}$.

Definition 9 (Event-refined full-provenance influence). *The relation $\rightsquigarrow_{\text{FP}} \subseteq S \times S$ is generated over a trace (53) by the following dependencies.*

- (F1) Explicit data/designation influence. *If a value, reference, target designation, or heap value produced or obtained at e_i is a semantic input to the computation performed at a later event e_j , then $e_i \rightsquigarrow_{\text{FP}} e_j$.*
- (F2) Implicit control influence. *If the reachability of the command executed at e_j is control-dependent on a branch or loop condition evaluated at e_i , then $e_i \rightsquigarrow_{\text{FP}} e_j$.*

Its transitive closure is written $\rightsquigarrow_{\text{FP}}^+$.

The first generator exposes at event resolution the explicit influence that RGR propagate through expression provenance. The second exposes the implicit influence that RGR propagate through the PC stack. Event identifiers are *ghost annotations*: they do not occur in the premises of an RGR FPs reduction rule.

Proposition 3 (Conservative erasure). *Erasing event identifiers and $\rightsquigarrow_{\text{FP}}$ edges from an event-refined FPs run yields exactly the underlying RGR FPs reduction trace, with the same principal-valued labels, the same PC stack evolution, and the same accepted or blocked assignments.*

Proof. The refinement adds only ghost metadata to the already existing reduction steps. No original FPs premise reads that metadata. Erasure therefore commutes with one reduction step; induction over the length of (53) gives the result.

Lemma 1 (Full-provenance influence is causally ordered). *For every event-refined FPs run,*

$$e_i \rightsquigarrow_{\text{FP}} e_j \implies e_i \prec_c e_j. \quad (54)$$

Consequently $\rightsquigarrow_{\text{FP}}^+ \subseteq \prec_c$.

Proof. For explicit influence, the value/reference or state consumed at e_j is produced or obtained at e_i . By dependency adequacy of the semantic causal order, producer/read events are ancestors of their consumers, hence $e_i \prec_c e_j$. For implicit influence, RGR lower the current pc only after the branch or loop condition has been evaluated and execute the controlled command while that lowered pc is on the stack. The reachability of the controlled event therefore depends on the condition event; dependency adequacy again gives $e_i \prec_c e_j$. These are exactly the two generators of $\rightsquigarrow_{\text{FP}}$. Transitivity of $\prec_c$ gives the closure statement.

The reverse inclusion is neither needed nor generally true. Two commands may be ordered by execution without the earlier one contributing data or control influence to the later one. Full provenance therefore induces an influence suborder on one common execution order; it does not create a second time.

10.2 Event-complete provenance and derived causal attribution

The sequential RGR calculus already closes all of its reads, writes, and control constructs inside one operational semantics. For a general machine execution we state the corresponding closure property directly.

Definition 10 (Closed event-complete execution). *A COSS execution is event-complete for a stated threat model when:*

- (E1) *every protected security-relevant effect is represented by an occurrence in S ;*
- (E2) *every explicit data/designation dependency and every implicit control dependency in scope is represented by $\rightsquigarrow_{\text{FP}}^+$; and*
- (E3) *every generating influence edge is causally sound as in Lemma 1.*

For a request-origin occurrence q and an effect occurrence e , define

$$\text{CausedBy}(e, q) \iff q \rightsquigarrow_{\text{FP}}^+ e. \quad (55)$$

This definition is event based, not stack based. Synchronous calls, shared state, a database, a queue, a timer, or an asynchronous callback do not require a different causal principle. When the later event consumes state or a message produced by the earlier event, the machine

semantics contains the corresponding dependency, for example

$$\text{write}_x \rightsquigarrow_{\text{FP}} \text{read}_x, \quad (56)$$

$$\text{enqueue}_q \rightsquigarrow_{\text{FP}} \text{dequeue}_q, \quad (57)$$

$$\text{setTimer} \rightsquigarrow_{\text{FP}} \text{fireTimer}, \quad (58)$$

$$\text{registerCallback} \rightsquigarrow_{\text{FP}} \text{invokeCallback}. \quad (59)$$

The wall-clock delay or disappearance of the original call stack does not erase causal ancestry. What matters is whether the modeled later occurrence depends on the earlier one.

Theorem 5 (Causal Attribution from full-provenance reachability). *In a closed event-complete execution,*

$$\text{CausedBy}(e, q) \implies q \prec_c e. \quad (60)$$

Hence causal attribution is a theorem of the event semantics rather than an independent assumption.

Proof. By Definition 10, $\text{CausedBy}(e, q)$ is $q \rightsquigarrow_{\text{FP}}^+ e$. Lemma 1 gives $\rightsquigarrow_{\text{FP}}^+ \subseteq \prec_c$. Therefore $q \prec_c e$.

10.3 Effect admission and authority continuity

Let the authority coordinate be $C(s) \subseteq O \times R$. An ordinary causal edge preserves PIC authority continuity:

$$s \rightarrow_c s' \wedge \text{Ord}(s, s') \implies C(s') \subseteq C(s). \quad (61)$$

At a multi-parent ordinary event the same rule is required for every immediate predecessor; equivalently, the new authority is contained in the intersection of the parent authority contexts. Any transition that legitimately broadens authority is not an ordinary continuation: it is an explicitly authorized origination, endorsement, composition, or relaxation event and starts a new authority bound, as in Section 15.

Let $\text{Eff} \subseteq S$ be the protected effect occurrences and let

$$\text{realize} : \text{Eff} \rightarrow O \times R \quad (62)$$

map an effect to the operation and actual resource it realizes. In the closed model, admission of a protected effect is part of the semantics:

$$e \in \text{Eff} \wedge \text{realize}(e) = (o, r) \wedge \text{Accepted}(e) \implies (o, r) \in C(e). \quad (63)$$

Thus complete mediation is internalized in the formal semantics: protected effects are precisely the events to which the admission rule applies.

Lemma 2 (Request-origin authority bound). *Let $q \prec_c e$ and suppose the causal path from q to e contains only ordinary authority-continuous edges. Then*

$$C(e) \subseteq C(q). \quad (64)$$

Proof. Choose a causal path $q = s_0 \rightarrow_c s_1 \rightarrow_c \dots \rightarrow_c s_k = e$. Equation (61) gives $C(s_k) \subseteq C(s_{k-1}) \subseteq \dots \subseteq C(s_0)$. Transitivity of set inclusion yields the claim.

Theorem 6 (Request-bounded effect theorem). *Let q be a request-origin occurrence and e an accepted protected effect in the same ordinary authority segment. In a closed event-complete execution,*

$$\text{CausedBy}(e, q) \wedge \text{realize}(e) = (o, r) \implies (o, r) \in C(q). \quad (65)$$

Proof. $\text{CausedBy}(e, q)$ implies $q \prec_c e$ by Theorem 5. Lemma 2 then gives $C(e) \subseteq C(q)$. Since e is accepted and realizes (o, r) , Equation (63) gives $(o, r) \in C(e)$. Therefore $(o, r) \in C(q)$.

10.4 Total model-level exclusion of the confused deputy

Definition 11 (Lineage-form confused deputy). *A confused-deputy state exists when there are a request-origin occurrence q , an effect $e \in \text{Eff}$, an executor D , and a privilege (o, r) such that*

$$\text{CausedBy}(e, q), \quad (66)$$

$$\text{realize}(e) = (o, r), \quad (67)$$

$$\text{exec}(e) = D, \quad (68)$$

$$(o, r) \notin C(q), \quad (69)$$

$$\text{IndependentAuthority}(D, (o, r)). \quad (70)$$

The final conjunct records the classical deputy situation: D independently possesses authority unavailable in the request context. It is not needed for the contradiction below.

Theorem 7 (Total confused-deputy exclusion in the closed COSS model). *Every closed event-complete COSS execution satisfying ordinary PIC authority continuity and protected-effect admission is free of lineage-form confused-deputy states on every ordinary authority segment.*

Proof. Assume a confused-deputy state satisfying (70). By its first two conjuncts and Theorem 6, $(o, r) \in C(q)$. The definition simultaneously requires $(o, r) \notin C(q)$, a contradiction. Independent authority held by D cannot enter the request's valid authority state through an ordinary edge because (61) forbids expansion.

The result is “total” in the same formal sense in which RGR’s Theorem 6 is unconditional for FPs: every execution admitted by the stated semantics satisfies the property. The two CDA definitions are not identified. RGR prove their extensional CDAF-E property for their calculus; Theorem 7 proves the lineage-form origin-authority property for the closed COSS event semantics. An explicit authorized endorsement or authority relaxation is represented by a distinguished causal event and starts a new segment; it is not a silent confused-deputy import

of authority. This mirrors RGR’s treatment of endorsed regions as intentional, rather than confused, use of lower-integrity influence.

The complete implication is

full-provenance influence on modeled events

$$\begin{aligned} &\Downarrow \\ &\text{CausedBy}(e, q) \Rightarrow q \prec_c e \\ &\Downarrow \\ &\text{PIC/COSS authority continuity} \\ &\Downarrow \\ &C(e) \subseteq C(q) \\ &\Downarrow \\ &\text{effect admission} \\ &\Downarrow \\ &\text{realize}(e) = (o, r) \Rightarrow (o, r) \in C(q) \\ &\Downarrow \\ &\neg \text{ConfusedDeputy}. \end{aligned}$$

(71)

11 Re-reading the Capability/IFC Tension

Consider the two-coordinate state

$$\Sigma = (C, F) \in X_{\text{auth}} \times X_{\text{ifc}}. \quad (72)$$

A decision that reads only C factors through the projection

$$p_C(C, F) = C. \quad (73)$$

If two states (C, F_1) and (C, F_2) require different information-flow verdicts, any C -invariant mechanism must nevertheless treat them identically. This is not a statement that capability systems are intrinsically incapable of information-flow enforcement. It is a statement about observational insufficiency: the decision cannot enforce a distinction represented only in a coordinate it does not observe.

This gives a compact reinterpretation of the historical sequence without turning it into a contest between security paradigms. Boebert’s oracle does observe clearance and classification when it grants a capability; the failure in his construction occurs in subsequent propagation. Even an oracle that correctly creates capabilities cannot, in the unmodified machine he defines, control their further propagation [4]. The distinction here is therefore between the grant decision and what later executions may acquire and exercise, not between a labeled and an unlabeled grant.

The later capability responses act directly on that propagation assumption. Miller and Shapiro’s data diode transmits data in one direction and capabilities in neither [6]; Miller, Yee, and Shapiro separately show how distinguishing capability transfer from data transfer blocks

the cross-level capability movement required by Boebert’s attack, while remaining within their reading of his definition [5]. Under a COSS encoding, an oracle grant that enlarges the carried authority would be an origination or explicitly authorized relaxation, whereas acquisition of a new capability during ordinary continuation cannot silently enlarge the authority coordinate. This is a COSS interpretation of the security distinction, not a claim that Boebert or the later capability papers used this state model.

Miller and Shapiro further show that program behavior and access abstractions can tighten authority beyond what an arrangement-only analysis of the protection graph predicts [6]. In COSS terms, these constructions are examples of mechanisms that make additional distinctions observable or restrict the semantic state on which the decision operates. Once those distinctions are observed, the mechanism no longer belongs to the projection-blind class covered by the impossibility corollary. The blindness claim concerns the information read when an event is admitted; it does not say that the original capability-granting oracle ignored clearance or classification.

The same reasoning generalizes. If

$$\Sigma = (S_1, S_2, \dots, S_m), \quad (74)$$

a policy that factors only through S_1 cannot enforce a property whose verdict varies with S_2 while S_1 is fixed. Security models can therefore be individually valid and still be jointly necessary.

12 PIC as a Derived Theory

The relationship between COSS and PIC is derivational rather than competitive.

PIC, first form. Take a single coordinate

$$X = X_{\text{auth}} = \mathcal{P}(O \times R). \quad (75)$$

Then COSS-validity is exactly PoR plus authority attenuation, and Product Continuity becomes PIC’s origin-binding result.

Relational PIC. Take

$$X = X_{\text{auth}} \times X_{\text{rel}}. \quad (76)$$

Then the product order is

$$(C', \rho') \preceq (C, \rho) \iff C' \subseteq C \wedge \rho' \Rightarrow \rho, \quad (77)$$

which is the product attenuation order already used by the relational theory [18].

General form. The natural closure is

$$X = \prod_{j \in J} X_j, \quad (78)$$

where authority, IFC, relational admissibility, provenance, geography, separation-of-duty memory, or another security system may contribute a coordinate. PIC is therefore not renamed or invalidated. It remains the authority-continuity specialization from which the more general product construction was discovered.

13 COSS in the PIC Series

COSS is the third step in a sequence of models that share the same basic shape: authorization is attached to an execution occurrence in causal history, PoR relates adjacent occurrences, a carried state is ordered by refinement, and ordinary continuation may preserve or restrict that state but not silently expand it. At the level of this continuity construction, each later paper contains the earlier state discipline as a specialization; this statement does not imply that the later construction reproduces every implementation theorem or deployment assumption of the earlier papers.

The containment statements are exact at the level claimed here. With $J = \{\text{auth}\}$, the COSS transition rule is PoR plus $C_{i+1} \subseteq C_i$, the validity rule of the first PoC model. With $J = \{\text{auth}, \text{rel}\}$ and relational refinement ordered by implication, the product order is the relational PIC attenuation order $(C', \rho') \preceq (C, \rho)$ iff $C' \subseteq C$ and $\rho' \Rightarrow \rho$ [18].

The separation results share the same factorization principle but should not be identified more strongly than their domains permit. For the authority-only event space, take $b = (o, r)$ and two histories ℓ, ℓ' such that $(o, r) \in C(\ell)$ and $(o, r) \notin C(\ell')$. The COSS lineage-forgetting corollary then gives the same required separation as the first paper’s lineage-invariance result. The relational paper subsequently shows that the earlier lineage-forgetting projection is one quotient instance of a more general factorization construction; its executor-profile quotients and COSS’s coordinate projections are applications of the same mathematical principle, not literally the same projection map [18].

The confused-deputy result is strengthened in the present paper. Hardy’s classical example motivates the problem of a deputy exercising authority for the wrong request context [16]. The first PoC paper excludes authority import as valid behavior inside a valid lineage through origin-bounded attenuation. The relational paper then isolates Complete Mediation and Causal Attribution as the premises needed to lift that closure to actual request-caused effects [18]. Section 10 internalizes those premises in a closed event semantics: protected effects are modeled admission events, and RGR-style full-provenance reachability derives causal attribution on the same occurrence order as PoR. The resulting Theorem 7 excludes every

	PoC [17]	Relational PIC [18]	COSS
Event domain	$O \times R \times L$	execution occurrences indexed by causal histories	$\mathcal{B} \times L$
Carried state	$C \subseteq O \times R$	$(C, [\rho]_{\text{sem}})$	$\Sigma \in \prod_j X_j$
Order	$\subseteq$	$\subseteq$ and $\Rightarrow$, componentwise	$\preceq_j$, componentwise
Continuity result	origin-bounded authority	Dual Monotonicity and Combined Closure	Product Continuity
Separation result	lineage-invariant policies cannot individuate occurrences	quotient factorization and observation-based indistinguishability	Projection Factorization and lineage-forgetting blindness

Table 1: The PIC series as a layered continuity construction.

lineage-form confused-deputy state along ordinary continuation in the closed model, while preserving explicit endorsement/authorized relaxation as a distinct causal transition class.

14 Heterogeneous Security-State Spaces

PIC already permits policy translation between heterogeneous operation/resource spaces [17]. The same idea extends componentwise.

At hop i , let coordinate j use state space $X_i^{(j)}$ with order $\preceq_i^{(j)}$. Let

$$T_{i \rightarrow i+1}^{(j)} : X_i^{(j)} \rightarrow X_{i+1}^{(j)} \quad (79)$$

be an order-theoretic policy translation. A translated transition satisfies

$$\sigma_{i+1}^{(j)} \preceq_{i+1}^{(j)} T_{i \rightarrow i+1}^{(j)}(\sigma_i^{(j)}). \quad (80)$$

If the translations are monotone and composable, define

$$T_{0 \rightarrow n}^{(j)} = T_{n-1 \rightarrow n}^{(j)} \circ \dots \circ T_{0 \rightarrow 1}^{(j)}. \quad (81)$$

Proposition 4 (Translated continuity). *If every $T_{i \rightarrow i+1}^{(j)}$ is monotone and every transition satisfies (80), then*

$$\sigma_n^{(j)} \preceq_n^{(j)} T_{0 \rightarrow n}^{(j)}(\sigma_0^{(j)}). \quad (82)$$

Proof. By induction on n . For $n = 0$ the claim is reflexivity. Suppose $\sigma_i^{(j)} \preceq_i^{(j)} T_{0 \rightarrow i}^{(j)}(\sigma_0^{(j)})$. Monotonicity of $T_{i \rightarrow i+1}^{(j)}$ gives $T_{i \rightarrow i+1}^{(j)}(\sigma_i^{(j)}) \preceq_{i+1}^{(j)} T_{0 \rightarrow i+1}^{(j)}(\sigma_0^{(j)})$, and (80) with transitivity gives the claim for $i + 1$.

Proposition 4 is order-theoretic: it requires monotonicity of the translations and proves a bound in the target orders. A stronger claim that a translation preserves the intended security meaning across heterogeneous behavior universes requires a separate semantic relation between those universes. This paper does not define such a cross-domain semantic soundness condition; an overly permissive semantic translation is therefore a policy or deployment error rather than a failure of the order-theoretic continuity theorem.

15 Ordinary Continuation versus Authorized Relaxation

Not every legitimate security transition is monotone. Declassification may relax an IFC constraint; a new authority grant may enlarge an authority state; a policy administrator may broaden an admissible geographic region.

COSS therefore distinguishes two classes of transition.

Definition 12 (Ordinary continuation). *An ordinary continuation is a PoR-linked transition satisfying $\Sigma_{i+1} \preceq \Sigma_i$ (or the translated form of (80)).*

Definition 13 (Authorized relaxation event). *An authorized relaxation event is an explicitly distinguished causal event whose validity is justified by a separate authority or policy rule and which may replace one or more coordinates by states not below their predecessors.*

The point is not to prohibit declassification, re-authorization, or policy change. It is to prevent those events from being confused with ordinary continuation. A security state may become more permissive only at a boundary whose authority to perform that relaxation is itself explicit and auditable.

This mirrors the distinction already present in PIC between attenuation within one lineage and the creation of new authority at an origin or explicit future composition boundary.

Relaxation events are still causal events: they are PoR-linked to their predecessor, and only the monotonicity conjunct is replaced by the separate authorization rule.

Proposition 5 (Segment continuity). *Let a PoR-linked lineage contain authorized relaxation events at indices $1 \leq r_1 < \dots < r_k$, where the event at r_m is the transition from s_{r_m-1} to s_{r_m} , all other transitions being ordinary continuations. For every i , let $r(i)$ be the largest $r_m \leq i$, or 0 if none exists. Then*

$$\Sigma_i \preceq \Sigma_{r(i)}. \quad (83)$$

The statement assumes a homogeneous state space along the segment. For segments containing translated hops, the corresponding bound is the one given by Proposition 4.

Proof. If $r(i) = i$, the claim is reflexivity. Otherwise, every transition leaving $s_{r(i)}$ and ending at or before s_i is an ordinary continuation, because $r(i)$ is the most recent relaxation-entry index. Product Continuity applied to the segment from $s_{r(i)}$ through s_i gives $\Sigma_i \preceq \Sigma_{r(i)}$.

Origin-boundedness is therefore replaced, in the presence of relaxation, by boundedness from the most recent explicitly authorized relaxation. A relaxation cannot be introduced by ordinary continuation; it can only restart the bound at an explicit, auditable point.

16 Causal DAGs, Joins, and Multiple Causes

The common causal substrate is a partial order, not inherently a linear call chain. Linear lineage notation denotes a path in that order. Fan-out, fan-in, shared state, asynchronous delivery, and multiple-cause effects are therefore represented by DAG structure before any security coordinate is considered.

For an ordinary event s with immediate predecessors $\text{Pred}(s) = \{p_1, \dots, p_k\}$, the conservative COSS rule is

$$\text{st}(s) \preceq \text{st}(p_m) \quad \text{for every } m \in \{1, \dots, k\}. \quad (84)$$

For the authority coordinate this specializes to

$$C(s) \subseteq \bigcap_{m=1}^k C(p_m). \quad (85)$$

Equation (84) is sufficient for Product Continuity along every parent path and for the request-bounded effect theorem even when an effect has several causes. In particular, if q is any cause whose influence reaches s , then the authority exercised at s remains bounded by $C(q)$ unless an explicit authorized relaxation/composition event intervenes.

A non-conservative composition policy may intentionally combine authority or other security states from several parents. Such a transition is not an ordinary continuation and is represented as an explicit authorized composition or relaxation event. Its policy semantics must state which origins justify the new state. This keeps the general theory honest without treating queues, databases, timers, callbacks, or fan-in as failures of causal attribution: those mechanisms contribute causal edges; only a genuine broadening of carried security state requires a separate authorization rule.

Miller and Shapiro’s “mutually suspicious composition” concerns the coexistence of independently motivated access abstractions over one capability graph [6]. The connection is conceptual rather than an identity of formal operators: COSS’s DAG rule is an event-level condition on how the security states of multiple causal parents may continue or be explicitly recomposed.

17 Related Work

Bell–LaPadula. Bell and LaPadula model secure system states together with rules of operation that preserve security under state transitions [2]. COSS does not replace those rules or claim that Bell–LaPadula is merely a set-inclusion policy. Rather, an effective Bell–LaPadula security state may serve as one coordinate when its local refinement relation and admitted-event semantics are specified; COSS adds the explicit causal-event axis and the product transport rule used when that state must coexist with other security states.

Denning’s lattice model. Denning organizes security classes as a lattice whose order is justified by the semantics of permitted information flow [3]. This is a direct example of an ordered security domain, but the IFC semantics remain Denning’s: COSS supplies neither the lattice nor a program analysis for discovering flows. It supplies a way to carry such an ordered state together with other coordinates along one causal execution.

Execution-history access control. Abadi and Fournet make run-time rights depend on execution history rather than only the current stack [7]. In their model, ordinary execution automatically intersects current rights with the static rights of code that runs, while an explicit special operation may modify current rights, at most restoring the static rights of that code and subject to further constraints. This is structurally close to the distinction here between ordinary non-expansive continuation and an explicit relaxation event. The models nevertheless have different domains: their construction computes a rights state for code execution history, whereas COSS abstracts over an arbitrary finite product of security-state coordinates and takes sound PoR as an explicit premise relating the modeled execution occurrences.

Decentralized information-flow control. Myers and Liskov define decentralized labels with owner-specific reader sets, a restriction order that forms a security-class lattice, and explicit declassification that is legal only when the process has authority to act for the relevant owner [8]. Their restriction/declassification distinction is therefore an important prior instance of monotone security restriction plus explicitly authorized relaxation. COSS treats such an IFC state as one possible coordinate; it does not replace the label semantics, acts-for relation, or static flow analysis of that work.

Rajani–Garg–Rezk full provenance. RGR give the first language-based extensional characterization of CDA-freedom, show that capability semantics does not prevent all implicit-designation CDAs, and introduce explicit-only and full provenance semantics [9]. FPs augments value provenance with a program-counter label stack so that branch and loop conditions contribute to the integrity

check on later writes. Their Theorem 6 proves CDA-freedom with endorsement unconditionally for their region calculus, via information-flow integrity and their Theorem 7. Read from COSS, FPs is a language-level instance of the influence relation required by Section 10. COSS does not reinterpret their theorem as a protocol result; the new step is a conservative event refinement that places FPs-style influence on the same execution occurrences as PoR and uses that common order to derive causal attribution for the lineage-form authority theorem.

Unified authorization and dependency calculi.

Flow-limited authorization integrates authorization and information-flow reasoning for delegation and trust [10]; the Dependency Core Calculus gives one calculus in which several dependency analyses, including information flow, are instances [11]; and operating-system DIFC designs track labels and privileges across processes [12, 13, 14]. These works unify security reasoning within a language semantics or an operating-system label model. COSS starts from a different object: authority bound to execution occurrences in a causal history that may span services and trust domains, with continuity verifiable at the receiving boundary. The approaches are complementary: a language-based or operating-system analysis can supply the influence relation, or a security coordinate, that COSS transports along PoR.

Capability systems and access abstractions. Miller, Yee, and Shapiro show that conclusions about capability systems depend on the capability model and on distinctions such as data transfer versus capability transfer, designation with authority, and access-controlled delegation channels [5]. Miller and Shapiro further show that security-enforcing program behavior and access abstractions can restrict authority beyond what a protection-graph-only analysis exposes [6]. COSS’s claim is not that these systems lack context, flow control, or causal reasoning. Its contribution is a uniform event/state construction and a factorization result: whenever a target verdict depends on a distinction erased by an observation, a decision invariant under that observation cannot reproduce the target verdict.

18 Semantic Closure and Realization Assumptions

The closed formal model now contains the two facts that earlier PIC work left as separate deployment assumptions. First, every protected effect is itself a modeled occurrence to which product admission is applied. Second, every security-relevant explicit data/designation dependency and implicit control dependency in the stated threat model is represented by the full-provenance influence relation. Theorem 5 then derives causal attribution from reachability.

This change is important for asynchronous systems. A queue, database, shared file, cache entry, timer, callback, or message broker does not break the causal chain merely because the original stack frame has ended. If the later event consumes state produced by the earlier event, the write/read, enqueue/dequeue, schedule/fire, or send/receive dependency is an edge of the same event DAG. The security state transported by COSS is therefore attached to causal occurrences rather than to synchronous call-stack lifetime.

A concrete realization must still justify that its evidence faithfully implements the formal semantics. In particular:

1. PoR evidence must be sound for the modeled immediate causal edges;
2. the runtime or analysis providing full provenance must conservatively expose every influence class claimed by the threat model;
3. each security coordinate and heterogeneous translation must faithfully represent the security meaning assigned to its refinement order; and
4. explicit endorsement, declassification, authority grant, or composition events must be separately authorized and observable as such.

These are realization obligations, not an additional Causal Attribution axiom. If a physical influence channel is omitted from the formal event semantics, no theorem about the model can silently cover that omitted channel.

19 Limitations and Open Questions

The causal-attribution item present in the earlier draft is discharged for the closed event model of Section 10. The remaining research tasks are narrower.

Semantic soundness of heterogeneous translations.

Translated Continuity proves preservation relative to the supplied ordered translation maps. Relating states and behaviors across different local vocabularies still requires a cross-domain semantic relation and a soundness condition for each translation.

Order soundness and coordinate choice. The meaning of $\preceq_j$ is domain-specific. A mathematically monotone but semantically wrong order proves the wrong property. Likewise, omitting a coordinate is sound only for target properties invariant under that omission; the projection theorem makes this requirement explicit.

Physical coverage. The formal theorem is total over the events and influence channels represented by the model. A real system must establish that the threat model’s relevant physical channels are represented by the

execution/provenance semantics. Covert or side channels not represented as causal influence remain outside the formal domain rather than counterexamples to the theorem.

Implementation evidence. The theory remains agnostic about whether evidence is carried in tokens, attestations, trusted-runtime state, gateways, proof-carrying objects, dynamic information-flow tracking, or another mechanism. Representation is secondary to whether the mechanism soundly establishes the common causal order and the security coordinates needed by the property being claimed.

20 Discussion

Scope of the framework. Across the three-paper sequence, the common primitives are an execution occurrence, a causal history, PoR between adjacent occurrences, an ordered state carried by the history, ordinary continuation that does not expand that state, and an explicit transition class for authorized relaxation. The continuity and separation results arise from these primitives plus the stated semantic and enforcement assumptions. The order theory used here is elementary; the contribution claimed is the choice of authorization domain and the causal transport discipline, not a new result in order theory.

The proposed shift is small syntactically but large semantically. Instead of asking one security model to become universal, COSS treats each model as a coordinate or influence view over the same causal execution. The RGR integration makes the distinction concrete: the IFC policy state is one coordinate, while full provenance is an influence suborder on the same occurrences. Neither requires a second temporal axis.

The general invariant is:

Ordinary causal transport may not make any carried security state more permissive according to that security system’s own refinement order. Any relaxation must be an explicit, separately authorized causal event.

This separates three concerns that are often conflated:

1. *what* a security system considers more or less permissive;
2. *which* security systems a decision must observe; and
3. *whether* the observed state belongs to the causal execution that is actually producing the effect.

Capability authority, Bell–LaPadula-style information-flow constraints, relational executor admissibility, provenance, geography, and other controls can therefore remain distinctly meaningful. They need not be merged into a single ontology. Their product is carried by the execution,

while PoR provides the causal axis that keeps the evolving states attached to one history.

An access abstraction need not always become a new coordinate. It may instead be the concrete mechanism that realizes a coordinate’s semantic extension, or it may restrict the admissible subset Y of product states. This is important for reading the capability literature correctly: the factory, Caretaker, and data-diode constructions of Miller and Shapiro constrain authority by program behavior, even when a coarser projection of the protection state would admit more possibilities [6]. COSS abstracts over the implementation choice and asks only which distinctions the receiving decision actually observes and preserves along the causal execution.

21 Conclusion

PIC placed authority-use events on a causal axis and made authority continuation non-expansive. COSS keeps that axis and generalizes the carried state from one authority set to a product of heterogeneous security states. The Product Continuity theorem is componentwise transitivity: ordinary causal transport cannot introduce a behavior forbidden by an origin state in any carried coordinate. The projection theorem gives the dual limitation: a decision cannot enforce a distinction that its observation erases.

Read from PIC, RGR provide an independent, language-level treatment of the confused deputy. Their capability semantics blocks explicit designation but not every implicit attack; their EP semantics tracks explicit provenance; and their FPs semantics adds program-counter provenance for implicit control flow. Their Theorem 6 proves CDA-freedom with endorsement unconditionally for their complete region calculus [9]. The present paper does not rewrite that theorem as PoR or PoC. Instead it lifts the influence represented by FPs to ghost event dependencies whose erasure leaves the original FPs execution unchanged.

This event lift resolves the causal question at the right level. A modeled machine execution has one causal order of occurrences. Authority, IFC policy, relational admissibility, and other security states are attached to those occurrences, while full provenance identifies which earlier occurrences influence later data or control. Full-provenance influence is therefore a suborder of the same execution order. Asynchronous state transfer through a database, queue, timer, callback, shared object, or message does not create a new time; it contributes additional causal dependency edges.

Once the modeled execution is closed under protected effects and full provenance, causal attribution is reachability. Combining that fact with ordinary PIC authority attenuation gives

$$\text{CausedBy}(e, q) \wedge \text{realize}(e) = (o, r) \implies (o, r) \in C(q). \quad (86)$$

The lineage-form confused-deputy condition requires the opposite, $(o, r) \notin C(q)$, and is therefore unsatisfiable along ordinary continuation. Independent authority held by the deputy cannot be imported through an ordinary causal continuation; intentional endorsement, authority grant, or non-conservative composition is a distinguished authorized event and begins a new authority bound.

Accordingly, the final model-level result is

closed event-complete full provenance + COSS/PIC authority continuity + protected-effect admission $\Rightarrow$ confused-deputy exclusion along ordinary continuation.
--

(87)

“Total” has the same formal reading as any semantic safety theorem: no execution admitted by the model contains the forbidden state. It does not claim security for physical influence channels omitted from the model. Within the stated event semantics, however, Causal Attribution is no longer an independent assumption; it is a derived theorem of the common causal order.

References

- [1] D. E. Bell and L. J. LaPadula, “Secure Computer Systems: Mathematical Foundations,” The MITRE Corporation, ESD-TR-73-278, Vol. I, 1973.
- [2] L. J. LaPadula and D. E. Bell, “Secure Computer Systems: A Mathematical Model,” The MITRE Corporation, ESD-TR-73-278, Vol. II, 1973.
- [3] D. E. Denning, “A Lattice Model of Secure Information Flow,” *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976, doi: 10.1145/360051.360056.
- [4] W. E. Boebert, “On the Inability of an Unmodified Capability Machine to Enforce the *-Property,” in *Proceedings of the 7th DoD/NBS Computer Security Conference*, Gaithersburg, MD, USA, Sep. 24–26, 1984, pp. 291–293. [Online]. Available: <https://csrc.nist.gov/files/pubs/conference/1984/09/24/7th-dodnbs-computer-security-conference/final/docs/1984-7th-conference-proceedings.pdf>.
- [5] M. S. Miller, K.-P. Yee, and J. S. Shapiro, “Capability Myths Demolished,” Technical Report SRL2003-02, Systems Research Laboratory, Department of Computer Science, Johns Hopkins University, Mar. 2003.
- [6] M. S. Miller and J. S. Shapiro, “Paradigm Regained: Abstraction Mechanisms for Access Control,” in *Advances in Computing Science – ASIAN 2003: Programming Languages and Distributed Computation*, Lecture Notes in Computer Science, vol. 2896, Springer, 2003, pp. 224–242, doi: 10.1007/978-3-540-40965-6_15.
- [7] M. Abadi and C. Fournet, “Access Control Based on Execution History,” in *Proceedings of the 10th Annual Network and Distributed System Security Symposium (NDSS)*, 2003.
- [8] A. C. Myers and B. Liskov, “A Decentralized Model for Information Flow Control,” in *Proceedings of the 16th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 129–142, 1997, doi: 10.1145/268998.266669.
- [9] V. Rajani, D. Garg, and T. Rezk, “On Access Control, Capabilities, Their Equivalence, and Confused Deputy Attacks,” in *2016 IEEE 29th Computer Security Foundations Symposium (CSF)*, Lisbon, Portugal, 2016, doi: 10.1109/CSF.2016.18.
- [10] O. Arden, J. Liu, and A. C. Myers, “Flow-Limited Authorization,” in *2015 IEEE 28th Computer Security Foundations Symposium (CSF)*, 2015.
- [11] M. Abadi, A. Banerjee, N. Heintze, and J. G. Riecke, “A Core Calculus of Dependency,” in *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 1999.
- [12] P. Efstathopoulos, M. Krohn, S. VanDeBogart, C. Frey, D. Ziegler, E. Kohler, D. Mazières, F. Kaashoek, and R. Morris, “Labels and Event Processes in the Asbestos Operating System,” in *Proceedings of the 20th ACM Symposium on Operating Systems Principles (SOSP)*, 2005.
- [13] N. Zeldovich, S. Boyd-Wickizer, E. Kohler, and D. Mazières, “Making Information Flow Explicit in HiStar,” in *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2006.
- [14] M. Krohn, A. Yip, M. Brodsky, N. Cliffer, M. F. Kaashoek, E. Kohler, and R. Morris, “Information Flow Control for Standard OS Abstractions,” in *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP)*, 2007.
- [15] L. Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978, doi: 10.1145/359545.359563.
- [16] N. Hardy, “The Confused Deputy (or Why Capabilities Might Have Been Invented),” *ACM SIGOPS Operating Systems Review*, vol. 22, no. 4, pp. 36–38, 1988.
- [17] N. Gallo, “Proof-of-Continuity: A Temporal Model for Authority Propagation in Distributed Systems and AI Agents,” arXiv:2607.08906, 2026.

- [18] N. Gallo, “Beyond Proof of Possession: A Relational Theory of Authority,” manuscript, 2026.